

Lab 3

ELECENG 3EY4

Alaa Abdelsalam – abdel74 – 400396373

Tina Ismail – ismait1 – 400299939

Marisa Patel – patem156 – 400393635

February 8, 2024

Table of Contents

Objective 1: Review of the Lab Manual	2
Contribution	2
Objective 2: Setting Up udev Rules.....	2
Question 1	2
Question 2	2
Question 3	2
Question 4	3
Question 5	3
Question 6	3
Question 7	4
Objective 3: Experiment with the Data Input and Output of the Lidar.....	4
Question 8	4
Question 9	7
Objective 4: Experiment with the Data Input and Output of the Camera.....	8
Question 10	8
Question 11	9
Objective 5: Experiment with the Data Input and Output of the Joystick	10
Question 12	10
Question 13	11
Question 14	18

Objective 1: Review of the Lab Manual

Contribution

Objective 2: Alaa

Objective 3: Tina

Objective 4: Marisa

Objective 5: Alaa, Tina, Marisa

Objective 2: Setting Up udev Rules

Question 1

To navigate to the rules, the command “cd /etc/udev/rules.d” was typed. This command uses the “cd” command. The “cd” command allows us to change the current directory. This command navigates to rules.d by starting from the root directory. Then, the directory is changed to the “etc” directory, which contains the configuration files and sub-directories. After this, the directory is changed to the “udev” directory, which is a device manager for the Linux kernel and is responsible for handling device nodes. Then, the directory is changed to the “rules.d” directory, which contains rules files used by the udev to control the behaviour of device nodes in the system. Therefore, by navigating to rules.d, it allows us to view and modify the rules as needed.

Question 2

To add the rule file for the lidar, the command “sudo gedit rplidar.rules” was typed. This command was typed in the terminal while being in the “rules.d” directory. This command uses the “sudo” and “gedit” commands. The “sudo” command grants us permission to run the “gedit” command to create and add the rule file for the lidar as the superuser. The “gedit” command used is a text editor, which allows us to create, open and view, and edit text files. The “gedit” command creates the “rplidar.rules” file and allows us to edit the file as well.

Question 3

To add the rule file for the VESC, the command “sudo gedit vesc.rules” was typed. This command was typed in the terminal while being in the “rules.d” directory. This command uses the “sudo” and “gedit” commands. The “sudo” command grants us permission to run the “gedit” command to create and add the rule file for the VESC as the superuser. The “gedit” command used is a text editor, which allows us to create, open and view, and edit text files. The “gedit” command creates the “vesc.rules” file and allows us to edit the file as well. This is the exact same procedure used to add the rule file for the lidar in Question 2.

Question 4

To add the udev rule file for the Logitech F710 joystick, the command “sudo gedit joystick-f710.rules” was typed. This command was typed in the terminal while being in the “rules.d” directory. This command uses the “sudo” and “gedit” commands. The “sudo” command grants us permission to run the “gedit” command to create and add the udev rule file for the Logitech F710 joystick as the superuser. The “gedit” command used is a text editor, which allows us to create, open and view, and edit text files. The “gedit” command creates the “joystick-f710.rules” file and allows us to edit the file as well. This is the exact same procedure used to add the rule files in Questions 2 and 3.

Question 5

To add the udev rule file for the RealSense camera, the command “sudo gedit 99-realsense-libusb.rules” was typed. This command was typed in the terminal while being in the “rules.d” directory. This command uses the “sudo” and “gedit” commands. The “sudo” command grants us permission to run the “gedit” command to create and add the udev rule file for the RealSense camera as the superuser. The “gedit” command used is a text editor, which allows us to create, open and view, and edit text files. The “gedit” command creates the “99-realsense-libusb.rules” file and allows us to edit the file as well. This is the exact same procedure used to add the rule files in Questions 2, 3, and 4.

Question 6

If you change a udev rule in the future, it is required to execute the commands “sudo udevadm control --reload-rules” and “sudo udevadm trigger” after making the changes and saving the files. The “sudo udevadm control --reload-rules” command is a reload command. It reloads the rules and ensures that any new changes made in the rules files are recognized and incorporated by the udev subsystem. When you make a change in the rules files, it is not immediately in effect, which is why a reload is necessary. A reload will allow udev to recognize these modifications. The “sudo udevadm trigger” command is a trigger command. It triggers udev, which is required in order to reapply the rules to the devices that are currently connected to the system. It reevaluates the currently connected devices against the modified rules. By using this command, it avoids the need to reboot the system as it ensures that the devices are properly configured based on the rules. It is required to apply these commands because both of these commands are valuable for managing and applying udev rules. These commands are required to be applied as they properly configure devices without having to restart the system. They ensure that changes made to the rules files take effect.

Question 7

It is required to edit the ROS launch files for the lidar. This is because it allows for the seamless integration into the ROS environment. These essential edits include configuring the parameters such as the serial port for accurate device communication, initializing nodes, and maintaining consistency throughout ROS deployments. Users can deploy the LiDAR node with the proper configurations with ease by specifying the required settings in the launch file. By doing this, the potential for errors while using the ROS environment are minimized.

Objective 3: Experiment with the Data Input and Output of the Lidar

Question 8

When moving the object, the numbers change. When the object is placed closest to the car, the number readings were very small being around 0.70. When the object is placed furthest from the car, the number readings were larger and around 3.0. When moving the object back and forth, the number readings are smaller when the object is close to the car, and the number readings are larger when the object is far from the car. When moving the object back and forth, the number readings are smaller when the object is close to the car, and the number readings are larger when the object is far from the car. The lidar sensor emits pulsed light waves, which bounce off of surfaces and back to the sensor, at which point a calculation can be computed to find the distance the pulse traveled based on the time the pulse took to return to the sensor. This means that when the object is placed closest to the car, the number readings will be small since the distance from the car to the object is small and therefore, it would take less time for the pulse to travel to the object and back to the sensor. In addition, when the object is placed furthest from the car, the number readings will be larger since the distance from the car to the object is larger and therefore, it would take more time for the pulse to travel to the object and back to the sensor. This explains how when moving the object back and forth, the values increase when the object is further away from the car and the values decrease when the object is closer to the car. This is how and why the numbers change.

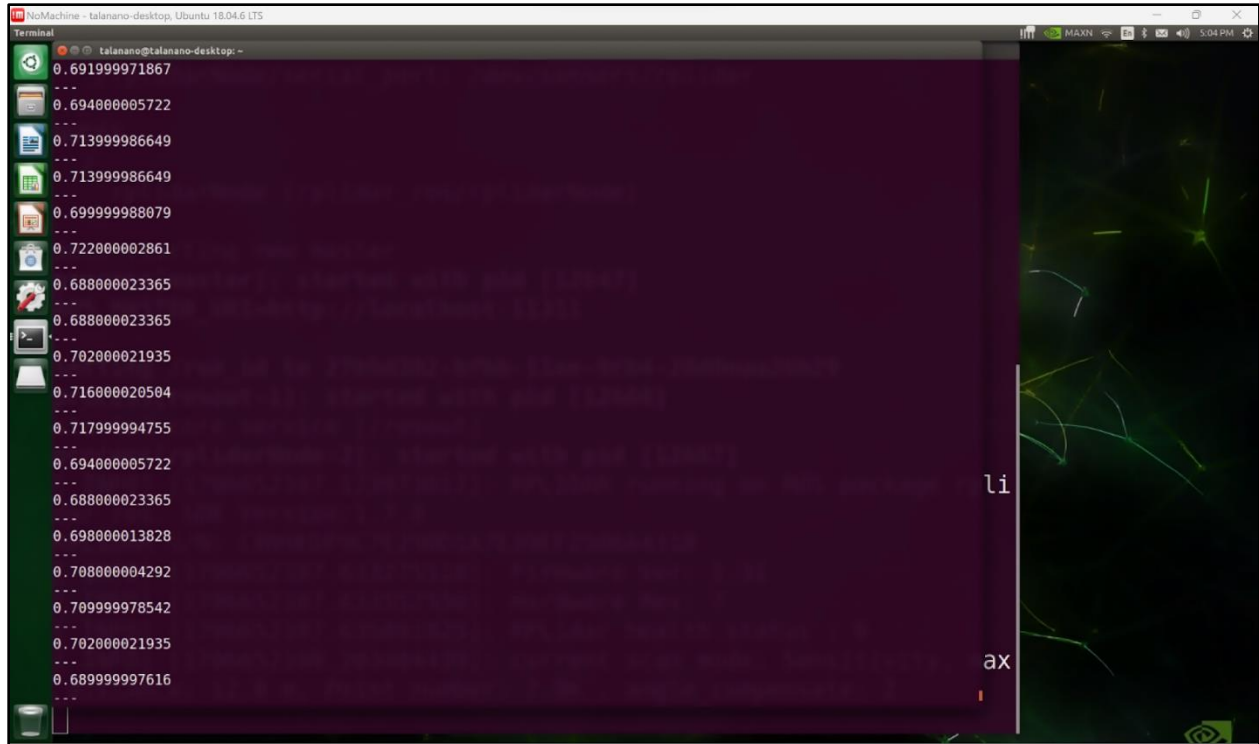


Figure 1. Array Ranges for when the Object is Closest to the Car

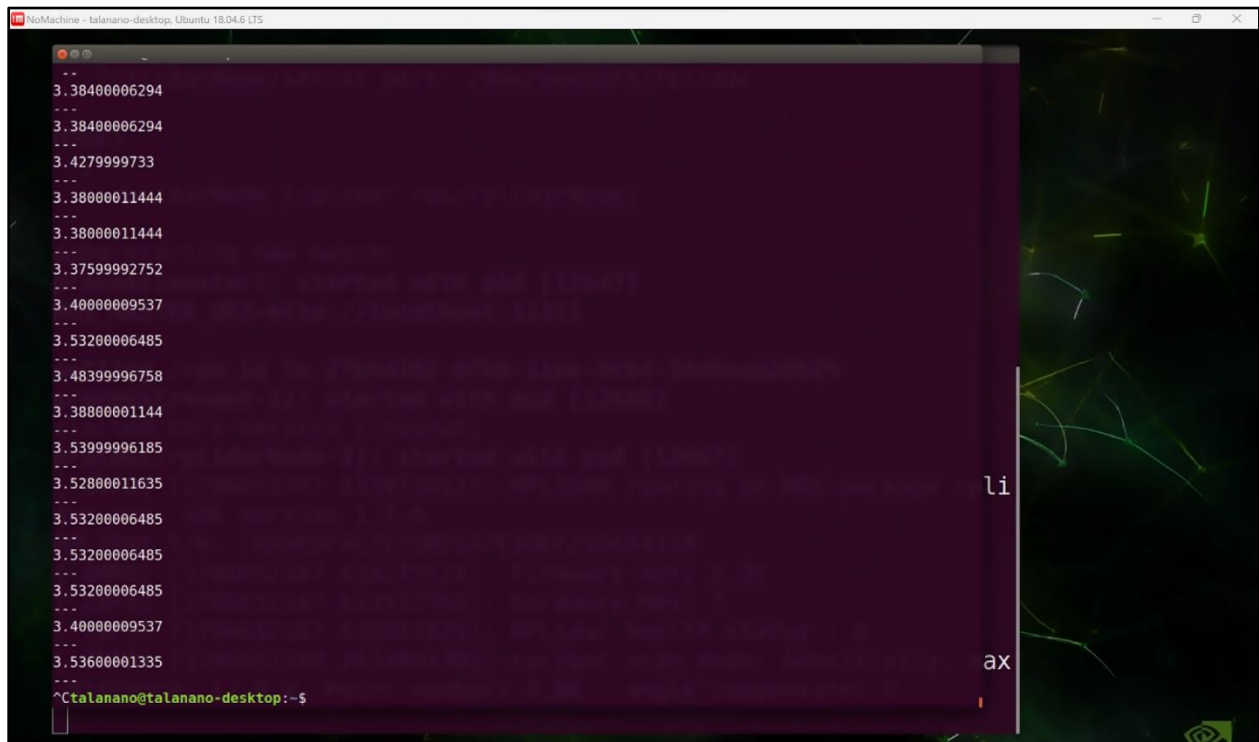


Figure 2. Array Ranges for when the Object is Furthest from the Car

```
Terminal
talanano@talanano-desktop: ~$
--- /rplidarNode/serial_port: /dev/sensors/rplidar
inf
---
0.435000002384
0.435999998951 rplidarNode (rplidar_ros/rplidarNode)
0.433999985456
--- starting new master
0.42199999094 process[master]: started with pid [12647]
0.407000005245 M_URI=http://localhost:11311
0.391000002623
0.370000004768 run_id to 27b5d302-bfbb-11ee-9cb4-28d0eaa26b29
0.34999999404 process[rosout-1]: started with pid [12660]
0.331000000238 process[rosout]: started with pid [12661]
0.312999993563 process[rplidarNode-2]: started with pid [12667]
3.55599999428 [1706652107.123973012]: RPLIDAR running on ROS package rplidar
3.55599999428 SDK Version:1.7.0
3.55599999428 S/N: C8B9EDF9C7E29BD1A7E39EF2586A431B
3.55599999428 [1706652107.633275518]: Firmware Ver: 1.31
3.55599999428 [1706652107.633557550]: Hardware Rev: 7
3.55599999428 [1706652107.635891825]: RPLidar health status : 0
3.56399989128 [1706652108.203404439]: current scan mode: Sensitivity, max
3.55599999428 e: 12.0 m, Point number: 7.9K , angle compensate: 2
```

Figure 3. Array Ranges for when the Object is Moved Back and Forth

Question 9

Figure 4 below displays the screenshot taken of the rviz page showing the point cloud map of the surrounding of the lidar. The emitted pulses from the lidar sensor generate point cloud data, which contains information about the location of objects. This point cloud data is used to create the point cloud map of the surroundings of the lidar. We observed a red map of the immediate surroundings of the lidar. This included the object that was used in Question 8. By moving the objects around the lidar, it was possible to observe the changes in the point cloud map in real time. By doing this, we were able to see the outline or shape of objects that we placed near the lidar.

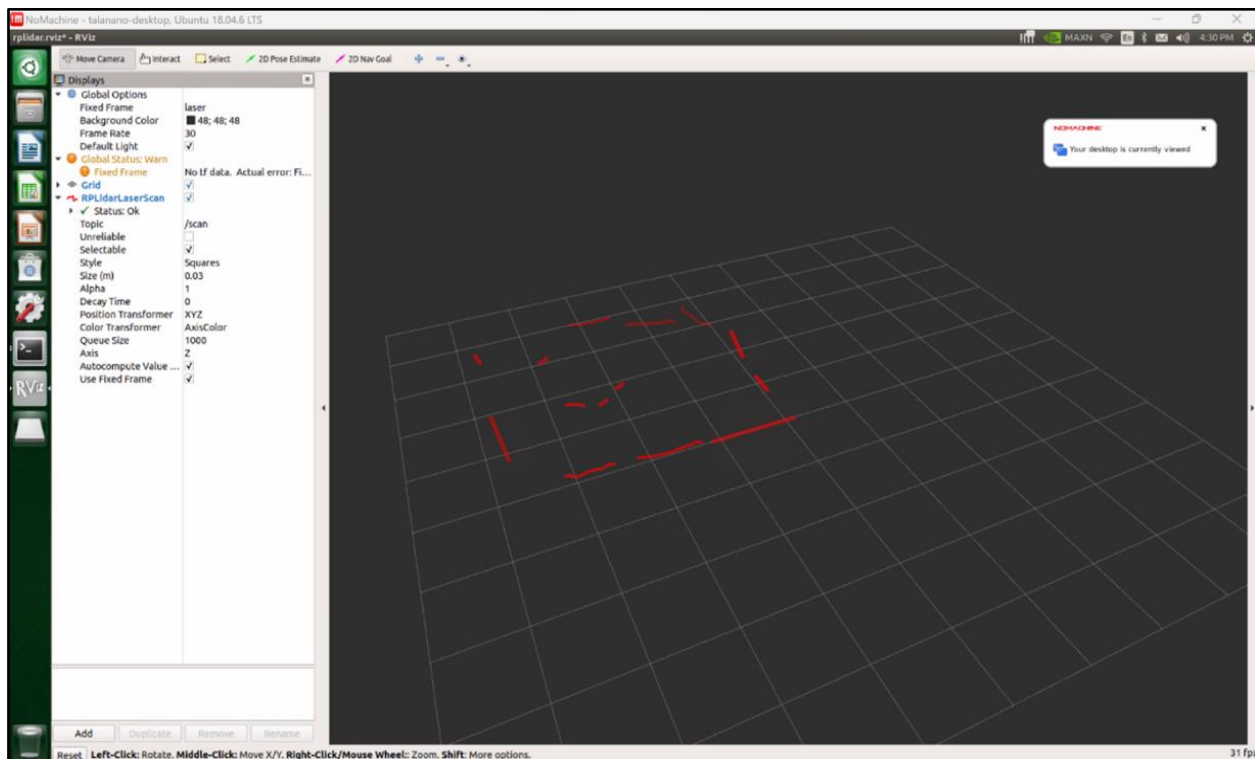


Figure 4. rviz Page Showing the Point Cloud Map of the Surrounding of the Lidar

Objective 4: Experiment with the Data Input and Output of the Camera

Question 10

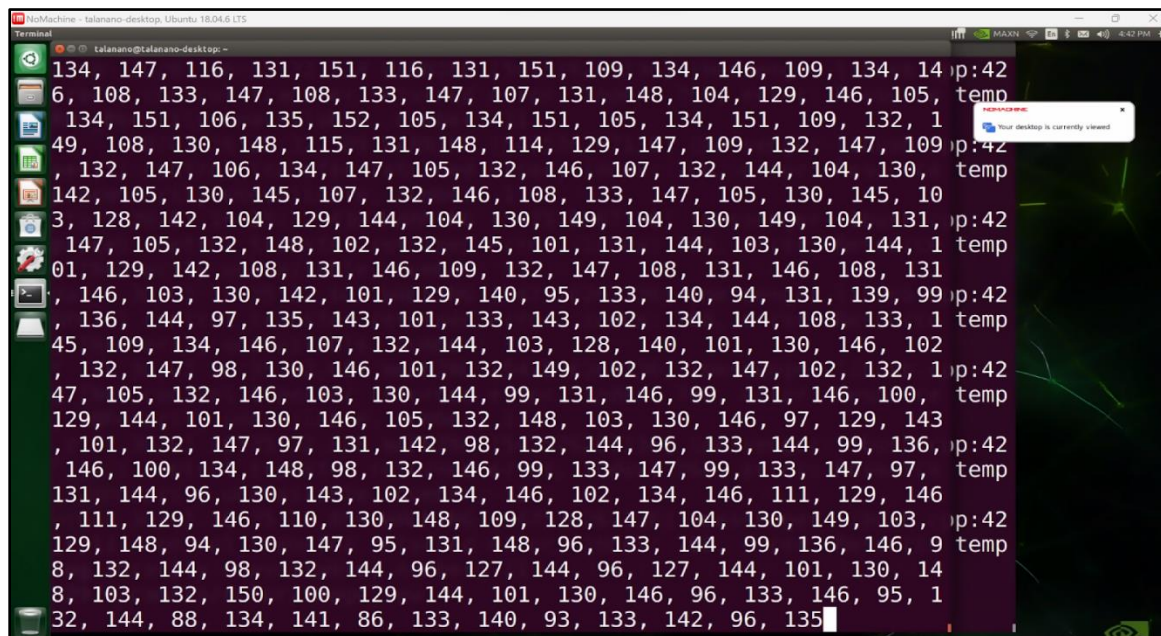


Figure 5. Output of `/camera/color/image_raw` with the 8-bit Unsigned Integer Range from 0 to 255

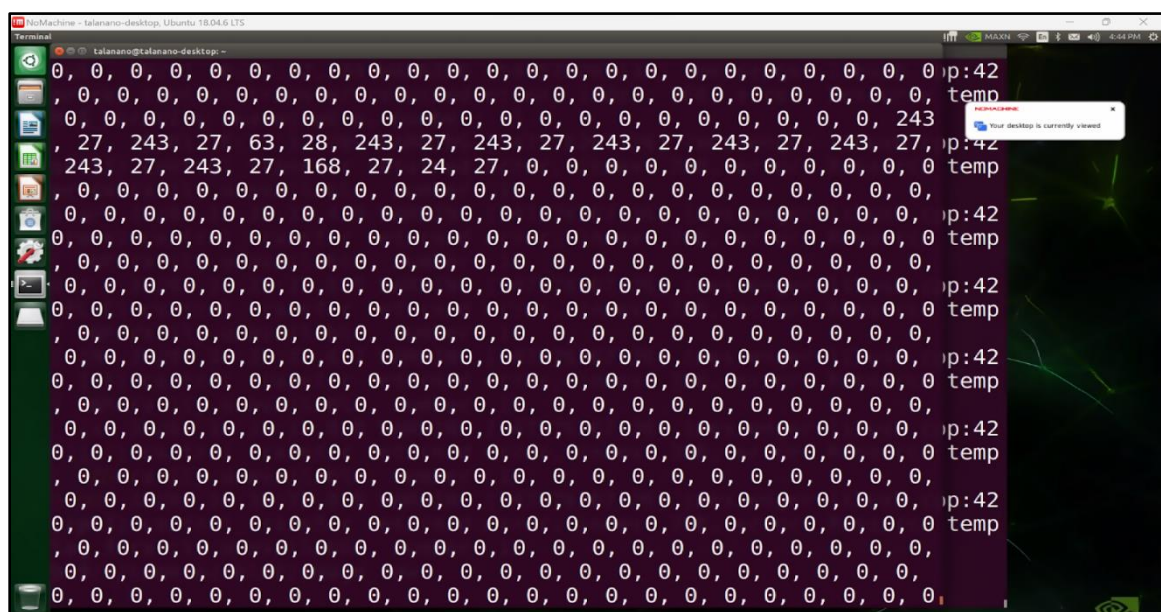


Figure 6. Output of `/camera/depth/image_rect_raw` with the 8-bit Unsigned Integer Range from 0 to 255

Question 11

The depth camera represents the raw coloured image since they are both produced from the same camera module/sensor. The main difference is that the raw coloured image represents an image in the traditional sense using RGB values and painting a scene (digitally speaking), while the depth image represents the same exact scene in a different way, providing additional information about how far away objects in the scene are. The depth camera represents the depth of each pixel in the raw coloured image, which provides a grayscale representation of the distance from the camera to the objects present in the scene. The grayscale appearance shows the depth as objects closer to the camera are darker in the grayscale and objects further away from the camera are lighter. This is how the depth camera represents the raw coloured image.

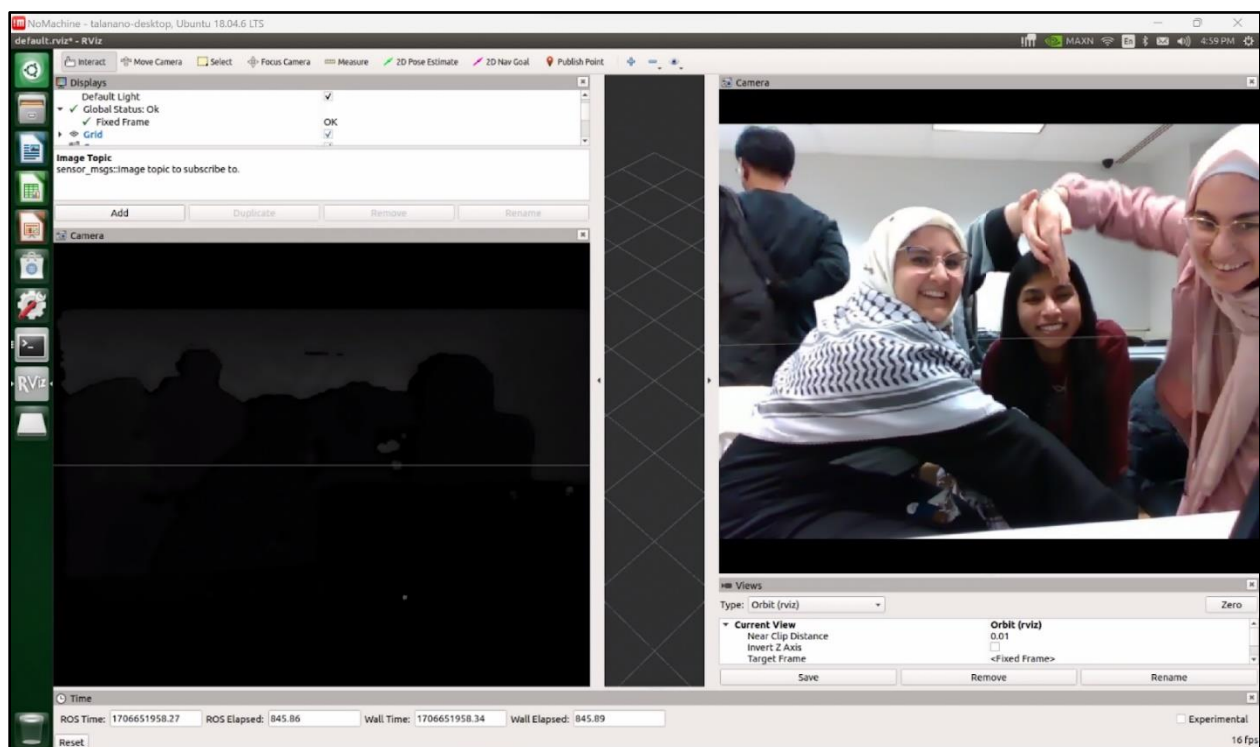


Figure 7. Camera and Depth Camera

Objective 5: Experiment with the Data Input and Output of the Joystick

Question 12

`<launch>`: This marks the beginning of the launch file and the start of the ROS launch configuration.

`<node name="joy_node" pkg="joy" type="joy_node">`: This defines a ROS node called "joy_node" that belongs to the "joy" package and are set in the type of "joy_node." This node allows interfacing with joystick toggles and publishing the input data received as ROS messages.

`<param name = "dev" value = "/dev/input/js0" />`: This sets a parameter called "dev" with a value of '/dev/input/js0'. It specifies the device path file of the joystick showing the location where the node listen for input from the joystick hardware.

`<param name = "autorepeat_rate" value = "2" />`: This sets a parameter called "autorepeat_rate" with a value of "2". This shows the rate at which the joystick node publishes input messages, when the joystick's toggle position remains unchanged. Here, the value is set to '2Hz' which indicates that two sets of messages are being transmitted per second.

`<param name = "coalesce_interval" value = "0.05" />`: This sets a parameter called "coalesce_interval" with the specified value of "0.05" in seconds. This indicates where messages are coalesced or combined together before it is published. The smaller the value is, the higher improvement in the response time of the vehicle to the input commands, and the lower the delay time between input updates.

`</node>`: This marks the end of the "node" block, which closes the "joy_node" block ROS node.

`</launch>`: This marks the end of the launch file, indicating the launch file configuration to be closed.

Question 13

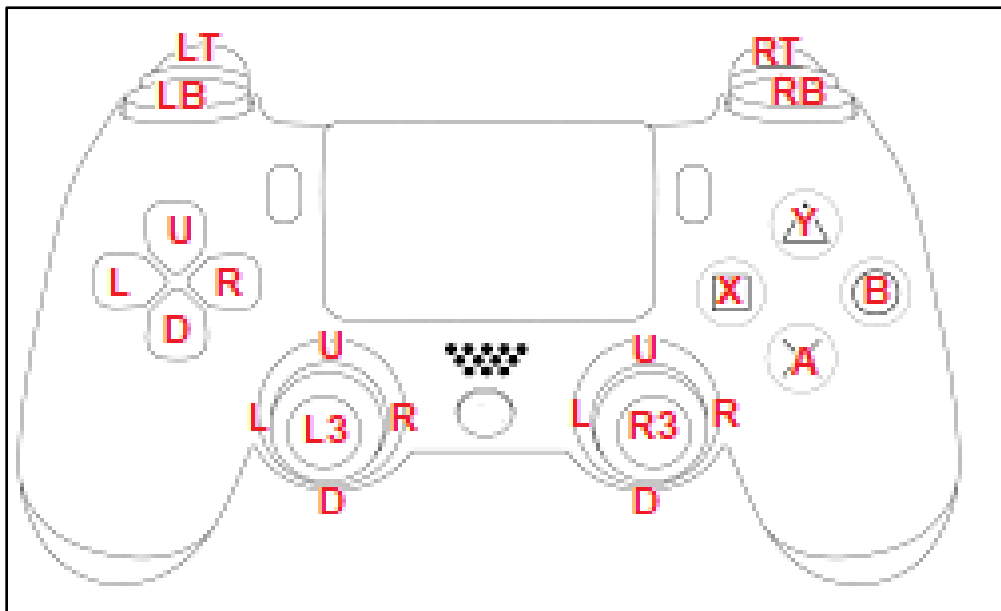


Figure 8. Controller Key Mapping for Reference

```
header:
  seq: 518
  stamp:
    secs: 1706650912
    nsecs: 796410061
  frame_id: "/dev/input/js0"
axes: [0.0, 0.0, -1.0, -0.0, -0.0, 0.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 9. LT

```
header:
  seq: 519
  stamp:
    secs: 1706650913
    nsecs: 186342821
  frame_id: "/dev/input/js0"
axes: [0.0, 0.0, 1.0, -0.0, -0.0, 0.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 10. LB

```
header:
  seq: 520
  stamp:
    secs: 1706650913
    nsecs: 410881258
  frame_id: "/dev/input/js0"
axes: [0.0, 0.0, 1.0, -0.0, -0.0, 0.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
```

Figure 11. RT

```
header:
  seq: 526
  stamp:
    secs: 1706650916
    nsecs: 186838236
  frame_id: "/dev/input/js0"
axes: [0.0, 0.0, 1.0, -0.0, -0.0, 0.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 12. RB


```
header:
  seq: 527
  stamp:
    secs: 1706650916
    nsecs: 256885945
  frame_id: "/dev/input/js0"
axes: [0.0, 0.0, 1.0, -0.0, -0.0, -1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 13. U

```
header:
  seq: 532
  stamp:
    secs: 1706650918
    nsecs: 737706569
  frame_id: "/dev/input/js0"
axes: [0.0, 0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 14. R

```
header:
  seq: 533
  stamp:
    secs: 1706650918
    nsecs: 955064746
  frame_id: "/dev/input/js0"
axes: [0.0, 0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
```

Figure 15. D

```
header:
  seq: 539
  stamp:
    secs: 1706650921
    nsecs: 486645162
  frame_id: "/dev/input/js0"
axes: [0.0, 0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 16. L

```
header:
  seq: 540
  stamp:
    secs: 1706650921
    nsecs: 721111724
  frame_id: "/dev/input/js0"
axes: [-0.0, 1.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 17. Y

```
header:
  seq: 544
  stamp:
    secs: 1706650923
    nsecs: 525237140
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 18. B

```
header:
  seq: 545
  stamp:
    secs: 1706650923
    nsecs: 757150629
  frame_id: "/dev/input/js0"
axes: [-1.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 19. A

```
header:
  seq: 549
  stamp:
    secs: 1706650925
    nsecs: 581538129
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 20. X

```
header:
  seq: 550
  stamp:
    secs: 1706650925
    nsecs: 825698077
  frame_id: "/dev/input/js0"
axes: [-0.0, -1.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 21. L3 – U


```
header:
  seq: 554
  stamp:
    secs: 1706650927
    nsecs: 524895836
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 22. L3 – R

```
header:
  seq: 555
  stamp:
    secs: 1706650927
    nsecs: 729374065
  frame_id: "/dev/input/js0"
axes: [1.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 23. L3 – D

```
header:
  seq: 559
  stamp:
    secs: 1706650929
    nsecs: 709060627
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 24. L3 – L

```
header:
  seq: 569
  stamp:
    secs: 1706650933
    nsecs: 955006928
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 25. $R3 - U$

```
header:
  seq: 584
  stamp:
    secs: 1706650939
    nsecs: 467003384
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, 1.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 26. $R3 - R$

```
header:
  seq: 625
  stamp:
    secs: 1706650952
    nsecs: 824866191
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -1.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 27. $R3 - D$

```
header:  
  seq: 652  
  stamp:  
    secs: 1706650959  
    nsecs: 15124366  
  frame_id: "/dev/input/js0"  
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]  
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 28. R3 – L

Question 14

The difference between analog and digital signals of a controller is the difference in the representation of data. Analog signals create a smooth continuous spectrum of values, enabling variable inputs such as the joystick movements or pressure sensitivity on triggers, which means that analog sticks on a controller can detect changes in position to deliver precise and accurate control. Conversely, digital signals are binary and offer discrete values, representing clear on/off states or high/low like button presses. When a button is pressed, it generates a digital signal of either "on" (1) or "off" (0), which displays a discrete action taking place. Therefore, digital and analog signals serve specific functions in the controller input, where analog is continuous and digital is discrete values.